

OpenSeek-7 Jeopardy Answer Generation 技术报告

1. 任务概述

本代码面向 OpenSeek-7 Jeopardy 答案生成任务，目标是根据输入中的 `Category` 与 `Clue` 自动生成一个简短、规范的 Jeopardy-style 答案。与直接让模型回答不同，该代码采用“两阶段提示词推理流程”：

1. 第一阶段：路由分类

使用 Qwen 模型判断题目属于哪一种解题类型，即输出一级 `route` 和二级 `subroute`。

2. 第二阶段：答案生成

将第一阶段得到的 `route/subroute` 作为强制约束，重新构造答题 Prompt，让 Qwen 在指定解题视角下生成最终答案。

该设计的核心思想是：先把题目划分到更细的解题模式，再利用该模式约束答案生成，从而降低模型在 Jeopardy 题目中出现跑题、过度解释或答案不规范的问题。

2. 输入数据与输出格式

2.1 数据来源

代码通过如下路径读取 OpenSeek-7 数据集：

```
1 file_path = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-Annotation\data\openseek-7_jeopardy_answer_generation_all.json"
```

数据加载函数如下：

```
1 def task5_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数返回三类内容：

- `task_id`：任务编号；
- `examples`：带标准答案的训练/验证样例；
- `test_samples`：需要生成答案的测试样例。

2.2 单条输入格式

代码默认每条样例的 `input` 字段包含类似如下结构：

```
1 Category: SOME CATEGORY
2 Clue: SOME JEOPARDY CLUE
```

其中：

- `Category` 表示 Jeopardy 题目类别；
- `Clue` 表示需要回答的题干。

代码通过 `parse_input_text()` 解析这两个字段。

2.3 输出格式

当 `eval_flag=True` 时，代码处理测试集并输出 JSONL 文件，每行格式如下：

```
1 {"test_sample_id": "xxx", "prediction": "answer"}
```

该格式符合一般提交文件要求，即每条测试样例对应一个 `test_sample_id` 和一个模型预测答案 `prediction`。

3. 整体流程设计

代码主流程可以概括为：

```
1 读取 few-shot 路由样例
2      ↓
3 构造 ROUTE_PROMPT
4      ↓
5 读取 OpenSeek-7 数据集
6      ↓
7 对每条样例执行两阶段推理
8      ↓
9 第一阶段：Qwen 输出 route/subroute
10     ↓
11 解析 route/subroute JSON
12     ↓
13 第二阶段：构造 forced route/subroute 答题 Prompt
14     ↓
15 Qwen 输出 final_answer
16     ↓
17 本地验证或生成提交 JSONL
```

该流程中最重要的设计是：路由分类结果不会直接作为答案，而是作为第二阶段答题 **Prompt** 的约束条件。这样可以让模型先判断题目属于客观事实题、文字游戏题、补全题、拼写题还是多对象题，再根据对应模式生成答案。

4. Few-shot 路由样例读取

4.1 外部 few-shot 文件

代码首先读取外部 few-shot 文件：

```
1 FEW_SHOT_PATH = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-
  Annotation\rgs_q7\experiment\filtered_routes_output.txt"
2
3 with open(FEW_SHOT_PATH, "r", encoding="utf-8") as f:
4     few_shot_examples = f.read().strip()
```

该文件的作用是为路由分类阶段提供示例。由于 Jeopardy 题目大多数是客观知识问答，少部分才是文字游戏、补全、拼写等特殊类型，因此 few-shot 样例主要用于帮助模型稳定地区分不同路由类别。

4.2 few-shot 文件的必要性

如果只依靠短规则，模型容易把一些普通事实题误判成：

- `wordplay`
- `completion`
- `spelling`
- `multi`

而实际任务中大多数题目应优先归入 `factual`。因此，引入外部 few-shot 文件有三个作用：

1. 提供分类边界样例

通过人工筛选或模型生成的示例，告诉 Qwen 哪类题属于 `factual`，哪类题才属于 `wordplay/completion/spelling`。

2. 增强长上下文使用

将大量 few-shot 样例拼入 `ROUTE_PROMPT` 后，可以形成较长上下文，满足任务对长 Prompt 使用的要求。

3. 降低误路由概率

第一阶段路由越稳定，第二阶段答案生成越容易被约束到正确方向。

5. 第一阶段：路由分类 Prompt

5.1 ROUTE_PROMPT 的构造

第一阶段使用 `ROUTE_PROMPT` 作为 system prompt，其核心要求是：

```
1 你需要对 Jeopardy trivia 题目进行双层级路由分类：一级路由 route、二级子路由 subroute。
```

输出固定为：

```
1  {"route": "xxx", "subroute": "xxx"}
```

并且要求：

- 只输出纯 JSON；
- 不输出解释；
- 不输出思考过程；

- 默认优先判断为 `factual`；
- 只有在非常明确时才使用 `wordplay`、`completion`、`spelling`、`multi`。

5.2 一级路由设计

代码中定义了五类一级路由：

一级路由	含义	使用场景
<code>factual</code>	客观事实题	默认首选，适合绝大多数 Jeopardy 题
<code>wordplay</code>	文字游戏题	谐音、押韵、变位词、before-after 等
<code>completion</code>	补全题	谚语、名言、歌词、标题补全
<code>spelling</code>	拼写/形式约束题	首字母、尾字母、长度、字母模式
<code>multi</code>	多对象/集合题	多个实体、集合选择、组合类题

其中，`factual` 又细分为：

- `factual_person`
- `factual_place`
- `factual_title`
- `factual_number`
- `factual_definition`
- `factual_role_identity`
- `factual_bridge`
- `factual_list_or_set_selection`

这种二级分类使后续答题 Prompt 能更明确地引导模型，例如人物题偏向回答人名，地点题偏向回答国家/城市/地名，作品题偏向回答书名、电影名或歌曲名。

6. 第二阶段：答案生成 Prompt

6.1 PROBLEM_SOLVE_PROMPT

第二阶段使用 `PROBLEM_SOLVE_PROMPT`，其角色定义是：

```
1 | You are an expert Jeopardy-style trivia solver.
```

该 Prompt 明确要求模型输入：

- `category`
- `clue`
- `forced route`

- forced subroute
- instruction

并输出 JSON：

```
1 {
2   "reasoning_brief": "...",
3   "candidates": ["...", "...", "..."],
4   "final_answer": "..."
5 }
```

其中真正用于提交的是 `final_answer` 字段。

6.2 build_route_instruction()

为了让第二阶段更准确地使用路由结果，代码通过 `build_route_instruction(route, subroute)` 为不同路由动态生成说明。例如：

- 如果 `route=factual` 且 `subroute=factual_person`，则提示模型偏向人物识别；
- 如果 `route=factual` 且 `subroute=factual_place`，则提示模型偏向地点识别；
- 如果 `route=wordplay` 且 `subroute=homophone`，则提示模型偏向谐音推理；
- 如果 `route=completion`，则提示模型偏向短语、标题或引语补全；
- 如果 `route=spelling`，则提示模型关注首字母、尾字母、长度等形式约束。

该函数将分类结果转化为自然语言指令，使第二阶段不是简单地“知道分类”，而是能够按照分类采取不同解题策略。

7. JSON 解析与容错机制

7.1 路由结果解析

第一阶段模型理论上应输出严格 JSON：

```
1 {"route": "factual", "subroute": "factual_definition"}
```

但实际模型可能输出多余文本，因此代码先用正则提取 JSON 片段：

```
1 json_match = re.search(r'\{.*?\}', raw_str, re.DOTALL)
```

如果仍然失败，则用正则分别提取 `"route"` 和 `"subroute"`：

```
1 route_match = re.search(r'"route"\s*:\s*"([^"]+)"', raw_str)
2 subroute_match = re.search(r'"subroute"\s*:\s*"([^"]+)"', raw_str)
```

这种设计可以提升模型输出格式不稳定时的鲁棒性。

7.2 答案结果解析

第二阶段模型输出也可能不是严格 JSON，因此代码定义了：

```
1 def extract_json_from_text(raw_text: str) -> Dict[str, Any]:
```

该函数先尝试直接 `json.loads()`，如果失败，再用正则匹配 `{...}` JSON 块。最终通过：

```
1 final_answer_raw = data.get("final_answer", raw_str).lower()
```

提取最终答案。如果 JSON 解析失败，则直接使用原始模型输出作为答案。

8. 单样例处理流程

8.1 测试样例处理：process_single_sample()

`process_single_sample()` 用于正式生成测试集答案，其流程为：

1. 读取样例 ID 和输入文本；
2. 使用 `ROUTE_PROMPT` 调用 Qwen 进行路由分类；
3. 解析 `route/subroute`；
4. 解析 `Category/Clue`；
5. 构造第二阶段 payload；
6. 使用 `PROBLEM_SOLVE_PROMPT` 再次调用 Qwen；
7. 解析 `final_answer`；
8. 返回提交所需字段。

返回结构如下：

```
1 return {
2     "task_id": task_id,
3     "sample_id": sample_id,
4     "input": input_text,
5     "model_output": final_answer_raw,
6 }
```

8.2 训练样例验证：process_single_example()

`process_single_example()` 与 `process_single_sample()` 基本一致，但额外读取标准答案：

```
1 gt = sample["output"][0]
```

并返回：

```
1 | "predict_right": gt == final_answer_raw
```

该函数用于在本地 examples 上估计模型方案效果。代码主程序在 `eval_flag=False` 时默认运行该路径，并统计前 1000 条样例的正确率。

9. 并发处理设计

代码使用 `ThreadPoolExecutor` 进行并发调用：

```
1 | max_workers = 200
2 | with ThreadPoolExecutor(max_workers=max_workers) as executor:
3 |     for result in tqdm(executor.map(process_func, data), total=total_cnt,
4 |                           desc="Evaluating"):
5 |         results_list.append(result)
```

这里使用的是 `executor.map()`，而不是 `as_completed()`。它的好处是：

- 并发执行；
- 返回结果顺序与输入顺序一致；
- 便于后续按原始样例顺序写入 JSONL。

不过，`max_workers=200` 较大，如果 API 服务端限流或本地网络不稳定，可能导致大量请求失败。因此实际运行时可以根据服务器吞吐量改为 16、32 或 64。

10. 本地验证与正式提交

10.1 eval_flag=False：本地验证模式

默认代码中：

```
1 | eval_flag = False
```

此时程序运行：

```
1 | data = examples[:1000]
```

并计算：

```
1 | correct_rate = float(correct_cnt) / float(total_cnt)
```

该模式用于在训练/验证样例上检查 Prompt 设计是否有效。

10.2 eval_flag=True：正式生成模式

当设置：

```
1 | eval_flag = True
```

程序会处理 `test_samples`，并写出：

```
1 | out_path = r"...\\openseek-7-v1.jsonl"
```

输出 JSONL 格式：

```
1 | {"test_sample_id": "xxx", "prediction": "xxx"}
```